MECHATRONICS AND IOT

ASSIGNMENT - 5

DESIGN AND SIMULATE AN AIR POLLUTION MONITORING SYSTEM USING MQ2 GAS SENSOR.

TEAM MEMBERS

- **JAGADEEP S – 24MER18**
- **JEEVA P – 24MER019**
- **JOGINDER S – 24MER021**
- **REGUL R – 24MER046**

1. PROJECT OVERVIEW

The Air Pollution Monitoring System using MQ-2 Gas Sensor is a low-cost system developed to detect harmful gases and monitor the quality of surrounding air. Air pollution caused by smoke, gas leakage, and industrial emissions can create serious safety and environmental problems. This system provides a simple method for continuously monitoring the presence of harmful gases.

The MQ-2 gas sensor is the main sensing element of the system. It can detect gases such as LPG, methane, smoke, and other combustible gases. The sensor produces an electrical output according to the concentration of gases present in the environment. A microcontroller such as an Arduino Uno or ESP8266 reads this sensor output and analyzes the gas level.

When the detected gas level is within the normal range, the system indicates that the air condition is safe. If the gas level increases above the predefined threshold, the system activates an LED and buzzer to provide a warning. The measured values can also be displayed on an LCD or transmitted through Wi-Fi when an ESP8266 is used.

The system can be used in industries, workshops, laboratories, homes, and environmental monitoring applications. It helps provide early warning of gas leakage and poor air quality, improving safety and enabling continuous monitoring. The proposed system is simple, economical, and can be further developed into an IoT-based system for remote monitoring and data logging.

2. PROBLEM STATEMENT

Air pollution is a major concern in industrial, commercial, and residential environments due to the presence of harmful gases, smoke, and gas leakage. High concentrations of certain gases can affect human health, workplace safety, and the surrounding environment. Conventional air-quality monitoring systems can be expensive and may not be suitable for small-scale or low-cost applications.

Therefore, there is a need for a simple, economical, and reliable system that can continuously monitor harmful gases in the surrounding environment. The proposed Air Pollution Monitoring System uses an MQ-2 Gas Sensor to detect gases such as smoke, LPG, methane, and other combustible gases. The sensor output is processed by a microcontroller and compared with a predefined threshold value.

When the detected gas level exceeds the specified limit, the system provides an immediate warning through an LED and buzzer. The system can also be extended with an LCD display or IoT connectivity for real-time monitoring and data recording. This project provides an effective solution for basic air-quality monitoring and early detection of unsafe gas levels in industrial and environmental applications.

3. PROPOSE SOLUTION

The proposed solution is to develop a low-cost and real-time Air Pollution Monitoring System using an MQ-2 Gas Sensor and a microcontroller. The MQ-2 sensor continuously detects harmful gases and smoke present in the surrounding environment and produces an output based on the detected gas concentration.

The microcontroller reads and processes the sensor output and compares it with a predefined threshold value. When the gas level is within the safe range, the system indicates normal air conditions. If the gas concentration exceeds the set limit, an LED and buzzer are activated to provide an immediate warning.

The system can also be connected to an LCD display to show the gas level and pollution status. For advanced applications, an ESP8266 can be used to transmit the monitored data through Wi-Fi for remote monitoring. The proposed system is simple, economical, easy to implement, and suitable for industrial safety, environmental monitoring, and gas leakage detection.

The proposed system uses an MQ2 gas sensor integrated with an ESP8266 microcontroller to continuously monitor the surrounding air.

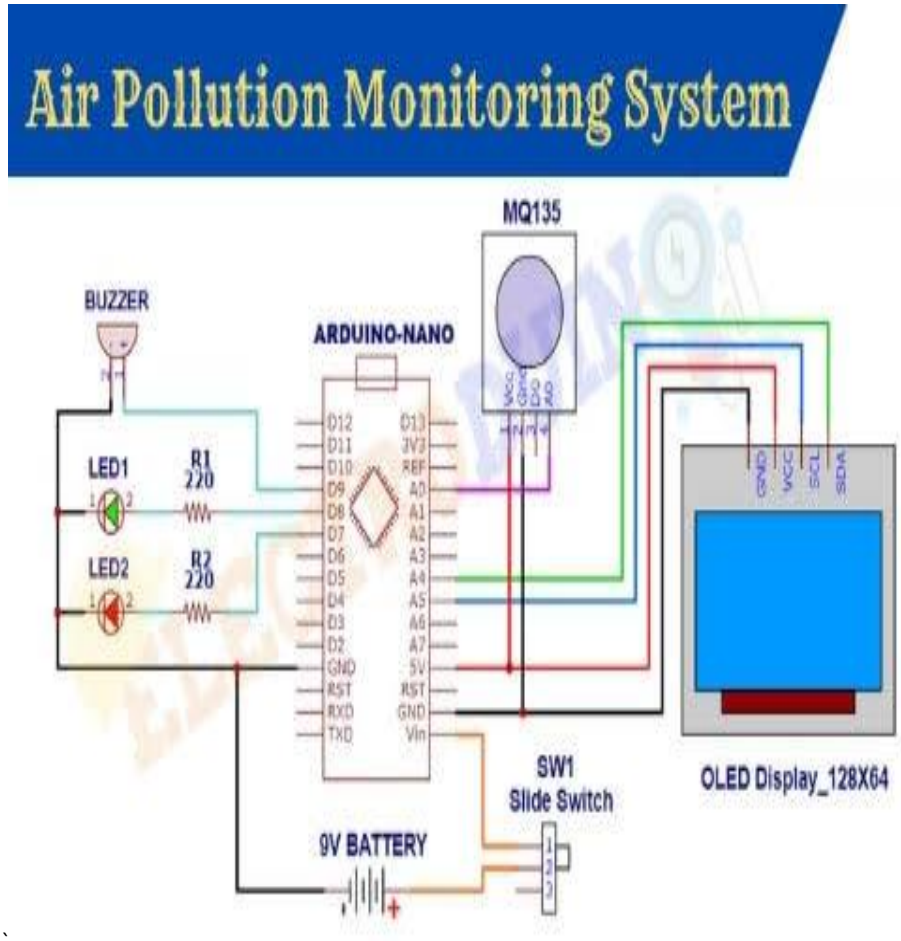
Working principle:

1. The MQ2 sensor senses smoke and combustible gases in the surrounding environment.
2. The sensor produces an analog output corresponding to the detected gas level.
3. The ESP8266 reads this analog signal through its ADC.
4. The ESP8266 processes the sensor reading and compares it with predefined threshold values.
5. The pollution level is classified into different categories such as:

- Safe
- Moderate
- Hazardous

6. If the detected level exceeds the hazardous threshold, an LED/buzzer alert is activated.
7. The system can optionally send the readings to an IoT/cloud platform through the ESP8266's Wi-Fi capability.
8. The readings can be displayed on a serial monitor, OLED/LCD display or web dashboard.

4. SYSTEM ARCHITECTURE



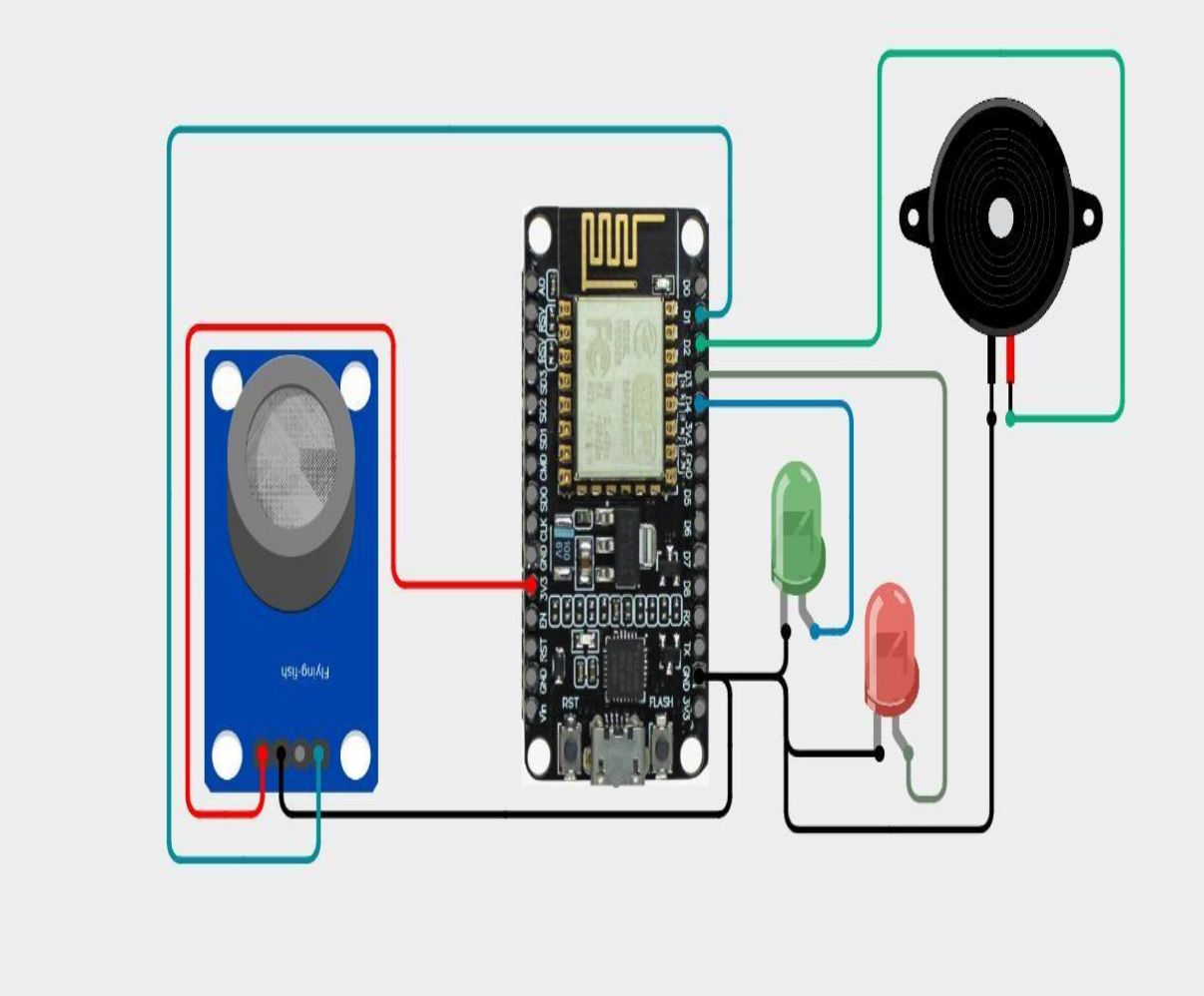
The Air Pollution Monitoring System consists of an MQ-2 gas sensor, microcontroller, alert unit, and display or monitoring unit. The MQ-2 gas sensor continuously detects harmful gases and smoke present in the surrounding air and produces an output according to the detected gas level. This output is sent to the Arduino or ESP8266 microcontroller, which processes the sensor data and compares it with a predefined threshold value. When the gas level is within the safe range, the system indicates normal air conditions. If the gas level exceeds the threshold, the microcontroller activates an LED and buzzer to provide an alert. The gas level and pollution status can also be displayed on an LCD or monitored remotely using IoT connectivity. The complete architecture provides continuous air-quality monitoring and early warning of unsafe gas levels.

5. CIRCUIT TABLE

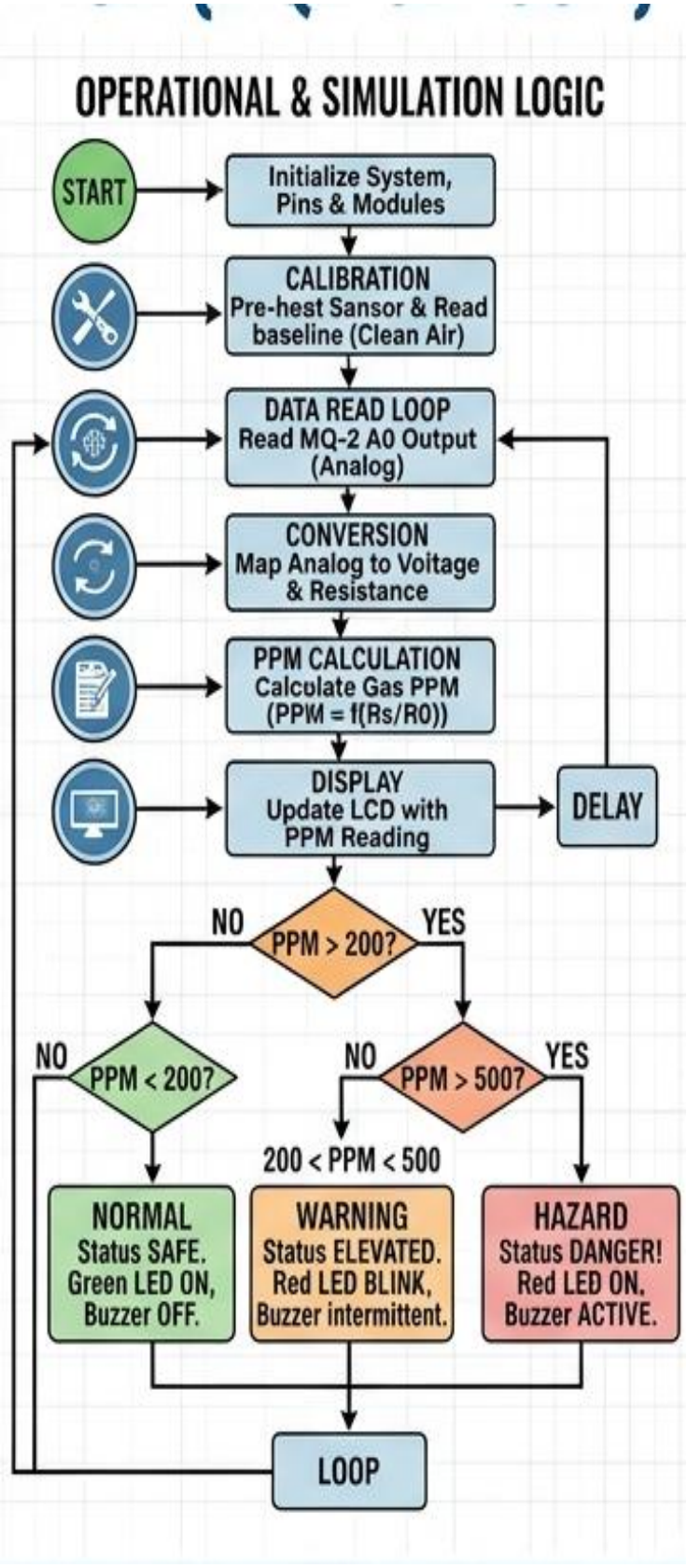
Component	Pin/Terminal	ESP8266 Connection	Purpose
MQ2 Gas Sensor	VCC	5V/IN	Power Supply
Green LED	Anode	GPIO 25 through 220 Ω	Safe indication
Red LED	Anode	GPIO 27 through 220 Ω	Hazardous indication
Buzzer	Positive	GPIO 14	Audible warning

The MQ-2 sensor is powered from the Arduino 5V supply, and its analog output is connected to A0. The Arduino continuously reads the sensor value. When the value exceeds the set threshold, the LED and buzzer are activated to indicate a high pollution or gas level.

6. CIRCUIT DESING



7. ALGORITHM


OPERATIONAL & SIMULATION LOGIC
START
Initialize System, Pins & Modules
CALIBRATION
Pre-hest Sansor & Read baseline (Clean Air)
DATA READ LOOP
Read MQ-2 A0 Output (Analog)
CONVERSION
Map Analog to Voitage & Resistance
PPM CALCULATION
Calculate Gas PPM (PPM = f(Rs/R0))
DISPLAY
Update LCD with PPM Reading
DELAY
NO
PPM > 200?
YES
NO
PPM < 200?
NO
PPM > 500?
YES
200 < PPM < 500
NORMAL
Status SAFE. Green LED ON, Buzzer OFF.
WARNING
Status ELEVATED. Red LED BLINK, Buzzer intermittent.
HAZARD
Status DANGER! Red LED ON, Buzzer ACTIVE.
LOOP

The proposed Air Pollution Monitoring System follows a continuous monitoring algorithm to detect harmful gases in the surrounding environment. Initially, the Arduino microcontroller and MQ-2 gas sensor are powered ON and initialized. The MQ-2 sensor is allowed to warm up and stabilize so that it can provide reliable readings. After initialization, the sensor continuously senses gases such as smoke, LPG, methane, and other combustible gases present in the air.

The analog output of the MQ-2 sensor is connected to the analog input of the Arduino. The Arduino periodically reads the sensor output and processes the received value to determine the gas concentration level. A predefined threshold value is stored in the program as the safe limit. The measured sensor value is compared with this threshold to determine the condition of the surrounding air.

If the sensor reading is below the threshold value, the system considers the air condition to be normal, and the warning LED and buzzer remain OFF. The system then continues to monitor the air. If the sensor reading exceeds the threshold value, the system identifies a high gas or pollution level. The Arduino activates the LED and buzzer to provide an immediate warning to the user.

The system continues to take sensor readings at regular intervals and updates the pollution status continuously. This repeated process allows the system to detect sudden changes in gas levels and provide an early warning. The algorithm can also be extended by adding an LCD display for showing sensor values or an ESP8266 module for transmitting the data to an IoT platform. Thus, the algorithm provides continuous, simple, and cost-effective monitoring of air pollution and harmful gas levels.

8. CODING

```
#include <ESP8266WiFi.h>
#include <ThingSpeak.h>
// ================================
// PIN DEFINITIONS
// ================================
#define gassensorPin A0

#define buzzPin 2
#define redlPin 5
#define greenlPin 4
#define DANGER_LEVEL 200
// ================================
// WIFI DETAILS
// ================================
const char* ssid = "Harish";
const char* password = "Harish@1234";
// ================================
// THINGSPEAK DETAILS
// ================================
unsigned long channelID = 3455657;
const char* writeAPIKey = "NAE0K4GXMAW2BMJ7";
WiFiClient client;
// ================================
// TIMER
// ================================
unsigned long previousMillis = 0;
const unsigned long thingSpeakInterval = 60000;
// ================================
// SETUP
// ================================
void setup()
```

```
{
  Serial.begin(9600);
  // Pin setup
  pinMode(gassensorPin, INPUT);
  pinMode(buzzPin, OUTPUT);

#define buzzPin 2
#define redlPin 5
#define greenlPin 4
#define DANGER_LEVEL 200
// ================================
// WIFI DETAILS
// ================================
const char* ssid = "Harish";
const char* password = "Harish@1234";
// ================================
// THINGSPEAK DETAILS
// ================================
unsigned long channelID = 3455657;
const char* writeAPIKey = "NAE0K4GXMAW2BMJ7";
WiFiClient client;
// ================================
// TIMER
// ================================
unsigned long previousMillis = 0;
const unsigned long thingSpeakInterval = 60000;
// ================================
// SETUP
// ================================
void setup()
{
```

```
Serial.begin(9600);
// Pin setup
pinMode(gassensorPin, INPUT);
pinMode(buzzPin, OUTPUT);
pinMode(redlPin, OUTPUT);
pinMode(greenlPin, OUTPUT);
digitalWrite(buzzPin, LOW);
digitalWrite(redlPin, LOW);
digitalWrite(greenlPin, LOW);
// ==============================
// WIFI CONNECTION
// ==============================
WiFi.mode(WIFI_STA);
WiFi.begin(ssid, password);
Serial.println();
Serial.print("Connecting to WiFi");
int attempts = 0;
while (WiFi.status() != WL_CONNECTED && attempts < 40)
{
  delay(500);
  Serial.print(".");
  attempts++;
}
Serial.println();
// ==============================
// CHECK WIFI
// ==============================
if (WiFi.status() == WL_CONNECTED)
{
  Serial.println("WiFi Connected!");
  Serial.print("IP Address: ");
```

```
    Serial.println(WiFi.localIP());

    Serial.print("Signal Strength: ");


    Serial.print(WiFi.RSSI());
    Serial.println(" dBm");
    // Start ThingSpeak
    ThingSpeak.begin(client);
  }
  else
  {
    Serial.println("WiFi Connection Failed!");
    Serial.println("Check WiFi name and password.");
  }
}
// ================================
// LOOP
// ================================
void loop()
{
  // ================================
  // READ GAS SENSOR
  // ================================
  int gasValue = analogRead(gassensorPin);
  // ================================
  // SAFE / DANGER
  // ================================
  int dangerStatus;
  if (gasValue > DANGER_LEVEL)
  {
    Serial.print("Gas Value: ");
```

```
    Serial.print(gasValue);
    Serial.println("   // Danger");
    Serial.print(WiFi.RSSI());
    Serial.println(" dBm");
    // Start ThingSpeak
    ThingSpeak.begin(client);
  }
  else
  {
    Serial.println("WiFi Connection Failed!");
    Serial.println("Check WiFi name and password.");
  }
}
// ================================
// LOOP
// ================================
void loop()
{
  // ================================
  // READ GAS SENSOR
  // ================================
  int gasValue = analogRead(gassensorPin);
  // ================================
  // SAFE / DANGER
  // ================================
  int dangerStatus;
  if (gasValue > DANGER_LEVEL)
  {
    Serial.print("Gas Value: ");
    Serial.print(gasValue);
    Serial.println("   // Danger");
```

```
  digitalWrite(redlPin, HIGH);
  digitalWrite(greenlPin, LOW);
  digitalWrite(buzzPin, HIGH);
  dangerStatus = 1;
 }
 else
 {
  Serial.print("Gas Value: ");
  Serial.print(gasValue);
  Serial.println("   // Safe");
  digitalWrite(redlPin, LOW);
  digitalWrite(greenlPin, HIGH);
  digitalWrite(buzzPin, LOW);
  dangerStatus = 0;
 }
 // ==============================
 // SEND TO THINGSPEAK EVERY 60 SEC
 // ==============================
 unsigned long currentMillis = millis();
 if (currentMillis - previousMillis >= thingSpeakInterval)
 {
  previousMillis = currentMillis;
  if (WiFi.status() == WL_CONNECTED)
  {
    Serial.println("--------------------");
    Serial.println("Sending data to ThingSpeak...");
    // Field 1 = Gas Value
    ThingSpeak.setField(1, gasValue);
    // Field 2 = Status
```

```
      // 0 = Safe
      // 1 = Danger
      ThingSpeak.setField(2, dangerStatus);
      int response = ThingSpeak.writeFields(channelID, writeAPIKey);
      if (response == 200)
      {
        Serial.println("ThingSpeak: Data sent successfully!");
      }
      else
      {
        Serial.print("ThingSpeak Error: ");
        Serial.println(response);
      }
      Serial.println("----------");
    }
    else
    {
      Serial.println("WiFi disconnected!");
    }
  }
  delay(500);
}
```

9. SYSTEM WORKING

The Air Pollution Monitoring System works by continuously detecting harmful gases and smoke present in the surrounding environment using the MQ-2 gas sensor. When the system is switched ON, the Arduino microcontroller and MQ-2 sensor are initialized. The MQ-2 sensor is allowed to warm up and stabilize before taking accurate readings.

The MQ-2 sensor detects gases such as LPG, methane, smoke, and other combustible gases. According to the concentration of gas present in the air, the sensor produces a varying analog output. This output is given to the analog input of the Arduino, which reads and processes the sensor value.

The Arduino compares the sensor reading with a predefined threshold value. When the reading is below the threshold, the system considers the air condition to be normal and the LED and buzzer remain OFF. When the reading exceeds the threshold, the system identifies a high gas or pollution level and activates the LED and buzzer as a warning.

The system continuously repeats this sensing and comparison process, allowing real-time monitoring of air quality. The system can also be extended with an LCD to display gas levels or with an ESP8266 to send the monitoring data through Wi-Fi. This makes the system useful for industrial safety, gas leakage detection, and basic environmental air-quality monitoring.

10. BILL OF MATERIALS

S. no	Component	Specification	Quantity	Cost
1	ESP8266	Node MCU ESP8266	1	Rs. 450
2	MQ2 Gas Sensor	Smoke/LPG/Methane detectio	1	Rs. 100
3	Green LED	5 mm	1	Rs. 20
4	Buzzer	Active Buzzer	1	Rs. 40
5	Breadboard	Standard 830-point	1	Rs. 100
6	Jumper Wire	Male-Male/Male-Female	1 set	Rs. 50
			Total	Rs. 760

11. PROTOTYPE IMPLEMENTATION

The prototype of the Air Pollution Monitoring System is implemented using an MQ-2 gas sensor, Arduino Uno, LED, buzzer, breadboard, and jumper wires. The Arduino Uno acts as the main controller of the system and processes the gas sensor readings. The MQ-2 sensor is connected to the Arduino to detect harmful gases and smoke in the surrounding environment.

First, the MQ-2 sensor is connected to the 5V and GND pins of the Arduino, while its analog output pin is connected to the A0 analog input. The LED is connected to a digital output pin to indicate the pollution condition, and the buzzer is connected to another digital output pin to provide an audible warning.

After making the connections, the Arduino is programmed using the Arduino IDE. The program continuously reads the analog value from the MQ-2 sensor and compares it with a predefined threshold. During normal air conditions, the LED and buzzer remain OFF. When the detected gas level exceeds the threshold, the Arduino activates the LED and buzzer to indicate a high pollution or gas level.

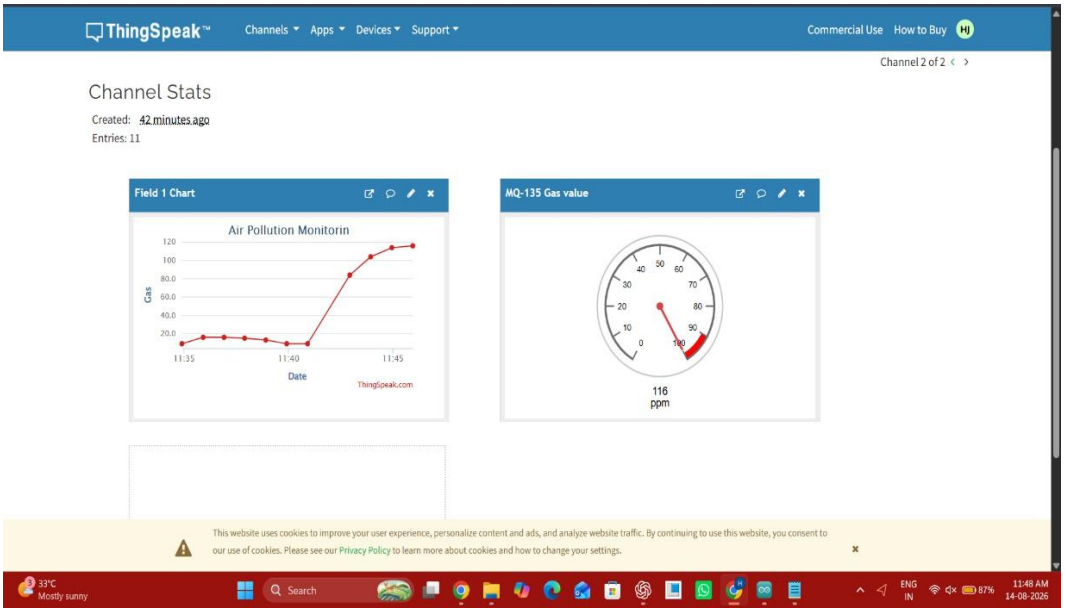
The prototype was tested by exposing the MQ-2 sensor to different levels of smoke or gas. The sensor output changed according to the detected gas concentration, and the warning system responded when the programmed threshold was exceeded. Thus, the prototype demonstrates the basic operation of a real-time air pollution and harmful gas monitoring system.

12. RESULT AND PERFORMANCE

- The developed Air Pollution Monitoring System using MQ2 Gas Sensor and ESP8266 was successfully implemented and tested under different gas and smoke conditions. The system continuously monitored the gas sensor output and classified the detected level into Safe, Moderate, and Hazardous conditions.
- The prototype demonstrated that the MQ2 sensor can effectively detect changes in the surrounding gas/smoke level, while the ESP8266 successfully processed the sensor output and generated appropriate warning indications.

The system provided:

- Real-time monitoring of gas and smoke levels.
- Fast response when an increase in gas concentration was detected.
- Three-level indication using green, yellow, and red LEDs.
- Green LED indication for safe air conditions.
- Yellow LED indication for moderate gas or pollution levels.
- Red LED indication for hazardous gas levels.
- Buzzer alert during hazardous conditions.
- Continuous monitoring without the need for manual checking.
- Reliable detection of changes in smoke and combustible gas levels.
- Low-cost and simple implementation using easily available components.
- Wi-Fi connectivity through ESP8266 for possible remote monitoring..


ThingSpeak™
Channels
Apps
Devices
Support
Commercial Use
How to Buy
Channel 2 of 2
Channel Stats
Created: 42 minutes ago
Entries: 11
Field 1 Chart
Air Pollution Monitorin
Gas
Date
ThingSpeak.com
MQ-135 Gas value
116
ppm
This website uses cookies to improve your user experience, personalize content and ads, and analyze website traffic. By continuing to use this website, you consent to our use of cookies. Please see our Privacy Policy to learn more about cookies and how to change your settings.
33°C
Mostly sunny
Search
ENG
IN
87%
11:48 AM
14-08-2026